

Four-Member GitHub Workflow — Start, Work in Periods, Review, Merge

This is the team's working sequence, not a script for manufacturing a commit history. A commit, review, test and completion date should be recorded only after the real action happens. Each member keeps one task branch and pushes completed work slices as the project progresses.

1. Repository setup (once)

Member 1 creates the real GitHub repository and adds Members 2, 3 and 4 as collaborators. The project should have one shared `main` branch and a `.gitignore` that excludes `.env`, `instance/`, `*.db`, `.venv/` and `venv/`.

If the project folder is not yet connected to Git, Member 1 initializes and pushes the baseline after reviewing what will be included:

```
git init -b main
git status --short
git add .
git status --short
git diff --cached
# Commit only after confirming secrets/local databases are excluded
git commit -m "chore: initialize sports registration project"
git remote add origin https://github.com/<OWNER>/<REPOSITORY>.git
git remote -v
git push -u origin main
```

Replace the placeholders with the real repository. Check ignored local files before the initial push:

```
git check-ignore -v .env instance/sports_registration.db .venv
```

Each person uses their own real GitHub account and sets their local author identity:

```
git config user.name "Your real name"
git config user.email "Your verified GitHub email"
```

2. Start a task branch (once per member)

Suggested branches:

```
Member 1: feature/backend-core
Member 2: feature/ui-refresh
Member 3: test/registration-workflow
Member 4: docs/demo-setup
```

Clone the actual repository once, then update `main` before branching:

```
git clone https://github.com/<OWNER>/<REPOSITORY>.git
cd <REPOSITORY>
git switch main
git pull --ff-only origin main
git switch -c <your-branch>
```

If the repository is already cloned, omit `git clone/cd`. Create the branch once only. Member 2's actual page-by-page periods are in `MEMBER_2_UI_UX.md`.

3. Repeat this work cycle for each completed code chunk

Stay on the same task branch for that member's related changes. Before starting the next chunk, confirm earlier changes are committed and the working tree is clean:

```
git status --short
git switch <your-branch>
```

If the team merged changes into `main` since the last period, bring them into the task branch deliberately:

```
git fetch origin
git merge origin/main
```

Then for the current task slice:

1. Work only on the agreed files/task for this period.
2. Run the app, tests or manual check that matches the change; note the actual result.
3. Inspect `git diff`; confirm no unrelated, secret or accidental route/form changes are included.
4. Stage the specific files, inspect the staged diff, then create one factual commit for a coherent completed change.
5. Push that task branch so the real progress is visible on GitHub. The first push sets the upstream; later period pushes update the same branch.

```
git status
git diff
git add <specific-files-for-this-slice>
git diff --cached
git commit -m "<type>: <factual summary of this completed change>"
git push -u origin <your-branch> # first push only
git push origin <your-branch>    # later period pushes
```

Examples of factual commit subjects when those changes have actually been made:

```
style: refine responsive site navigation
style: improve home and event listing layout
ui: clarify event details and registration choices
ui: improve registration form and confirmation states
style: improve admin dashboard and review tables
style: finish responsive and keyboard checks
```

These are examples, not pre-filled history. Do not make empty commits, backdate work, reuse someone else's author identity, split unchanged work just to increase commit count, or report an unrun test as passing. A real period may require no commit if no code has changed yet.

4. Pull request, review and merge (after the task is ready)

After the task branch contains the reviewed work, open a pull request with base `main` and compare the member's branch. The PR should truthfully list:

- what changed and which pages/files changed;
- the tests and manual checks actually run, with results;
- remaining issues, if any;
- genuine screenshots when the change is visual.

Assign a different member as reviewer. Resolve comments on the same branch, test again, commit any real fixes and push the updated branch. Merge only after review and team acceptance; do not push a task branch directly to `main`.

Suggested review pairing: Member 1 checks the UI/backend contract; Member 3 tests the merged flows; Member 4 follows the setup/demo docs; Member 2 checks the integrated screens.

5. After merge

Each member updates their local `main` and confirms the merged project still works:

```
git switch main
git pull --ff-only origin main
```

Evidence and work log

Use links/screenshots from the actual repository: branch, dated commits, PR, review discussion, test output and working app. Keep a brief actual work log for each period: date, task delivered, files, verification and commit/PR link. If something is planned but not complete, label it planned. Do not create evidence that implies work happened when it did not.